Herramientas Computacionales para Economistas

Universidad Nacional de La Plata

Clase 5

Belén Michel Torino

Temario de la clase

1. Objetivos de esta clase
2. Descargar con un click
3. Web scraping
4. PDF scraping
5. Base de datos relacionales (tipo SQL)
6. Conexión a una API

Table of Contents

1. Objetivos de esta clase
2. Descargar con un click
3. Web scraping
4. PDF scraping
5. Base de datos relacionales (tipo SQL)
6. Conexión a una API

Objetivos de la clase

- La clase de esta semana presenta distintas herramientas útiles para acceder a diversas fuentes de datos.
- Se busca exponer los paquetes más utilizados para hacer web scraping, pdf scraping, conexión a APIs, etc.
- Además, se conversará acerca de las dificultades que pueden aparecer en cada proyecto en particular y que estrategías pueden servir para sortearlas.
- Se espera que estas herramientas sean útiles para quienes esten pensando en sus proyectos de investigación/tesis.

Liberías principales

Algunas de las librerías más utilizadas para los propositos de esta clase son:

- **Requests:** Permite hacer solicitudes de HTTP
- **BeautifulSoup4:** permite extraer información de pagianas web
- **PyPDF2:** simplifica la manipulación de PDFs
- **Tabula:** extrae tablas de PDFs
- **Minecart:** facilita la extraccion de texto, imagenes, colores y formas de PDFs.
- **Sqlite3:** permite conectarse a bases de datos relacionales

Instalación

Para poder seguir esta clase a la par, con la consola o un Jupyter Notebook, por favor instalen las librerías indicadas en el documento que acompaña esta presentación.

Fuentes de datos

- PDF Scraping
- Web Scraping
- Descargar con un click (csv, excel)
- Conectarnos a una API
- Base de datos relacionales (tipo SQL)

Table of Contents

1. Objetivos de esta clase
2. Descargar con un click
3. Web scraping
4. PDF scraping
5. Base de datos relacionales (tipo SQL)
6. Conexión a una API

Descargar con un click

Con el paquete 'requests' podemos enviar solicitudes HTTP/1.1. para obtener el contenido de una url. Es como si estuvieramos accediendo a la url desde el navegador y descargando el contenido del archivo.

```
>>> def descarga_base_infectados(csv_url, csv_name):
        #Obtenemos el contenigo del archivo en el url
        #especificado
        req = requests.get(csv_url)
        url_content = req.content

        #Guardamos el contenido descargado en un archivo
        #con el nombre que elegimos en 'csv_name'
        csv_file = open(csv_name, 'w')
        csv_file.write(url_content.decode())
        csv_file.close()
```

Table of Contents

1. Objetivos de esta clase
2. Descargar con un click
3. Web scraping
4. PDF scraping
5. Base de datos relacionales (tipo SQL)
6. Conexión a una API

Web scraping: ¿Qué es?

- 'Web scraping' es la extracción de datos de sitios web.
- Es una herramienta para obtener información de forma programática y reproducible
- Utilizaremos web scraping cuando queramos datos de un sitio web que no tenga API
- Hay que se cuidadosos que nuestro web scraping no este violando ninguna regla de privacidad o del sitio
- La calidad de los datos que extraigamos va a depender de la calidad del sitio
- El navegador utiliza instrucciones en HTML para mostrar el sitio web.
- Detectamos el patrón en el HTML y definimos las secciones a extraer

Web scraping: ¿Qué es un HTML?

- HTML son las siglas de Hyper Text Markup Language
- HTML es el lenguaje de marcado estándar para crear páginas web
- HTML describe la estructura de una página web
- HTML consta de una serie de elementos que le dicen al navegador cómo mostrar el contenido.
- Los elementos HTML etiquetan partes de contenido para indicar "esto es un encabezado", "esto es un párrafo", "esto es un enlace", etc.

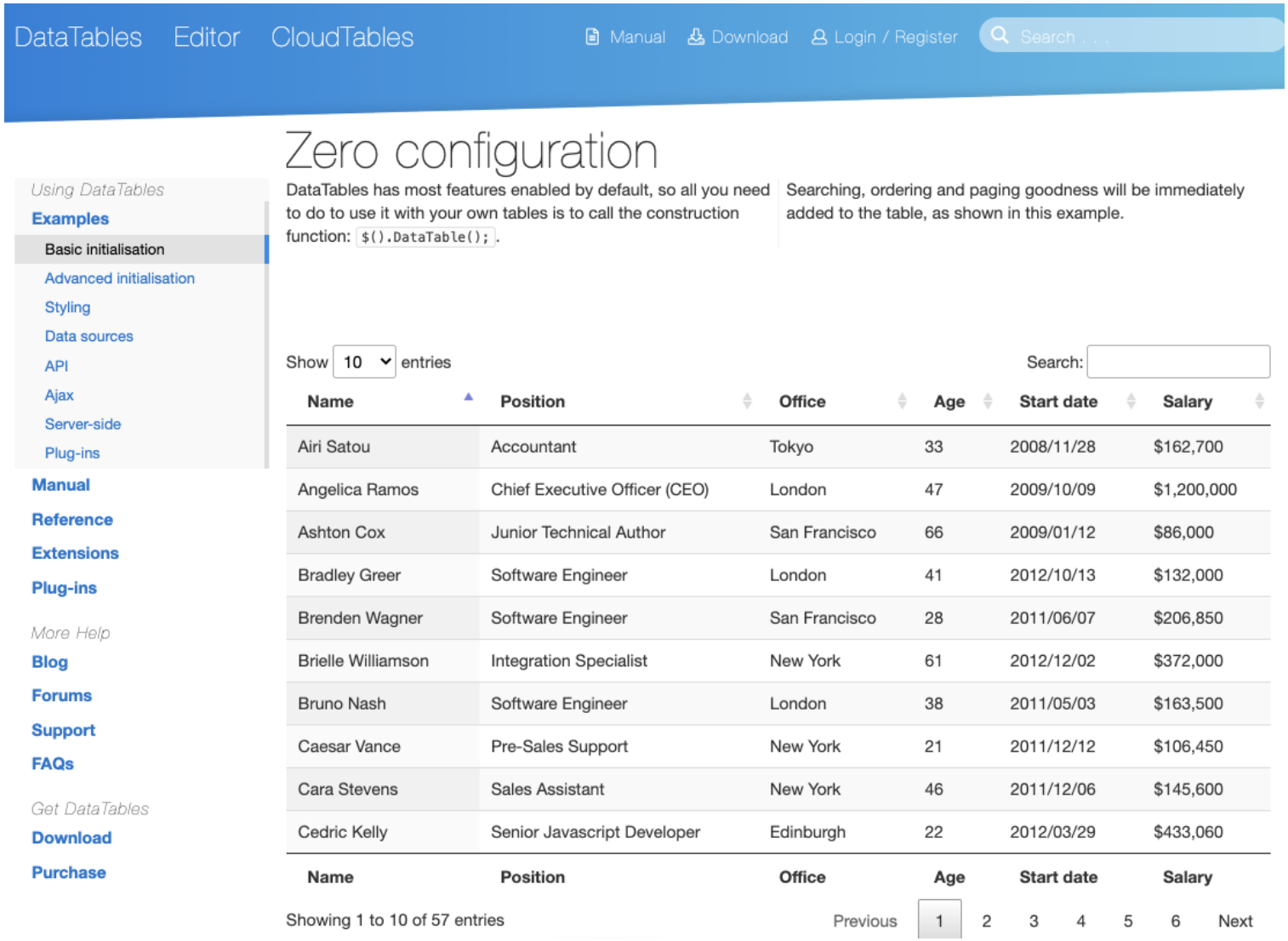
Web scraping

<!DOCTYPE>	Sirve para definir el tipo de documento
<html></html>	Define un documento HTML
<head></head>	Dentro se escribe información sobre el documento
<title></title>	Para escribir el título del documento. Se escriben dentro de <head>
<body></body>	Justo después de </head>. Es el contenedor del documento
, hasta	Para crear títulos de párrafos, de más importante <h1> a menos <h6>
	Los párrafos de texto de nuestro documento
 	Un salto de línea simple
<hr>	Salto de línea que indica un cambio de temática entre párrafos
<a>	Podremos escribir enlaces a otras páginas

fuente: `https://websgalicia.es/blog/etiquetas-html/`

Pueden consultar otras etiquetas (tags) en **CheetSheets de HTML**

Web scraping



Sitio: `https://datatables.net/examples/basic_init/zero_configuration.html`

Web scraping

```
<!DOCTYPE html>
<html>
▶ <head>…</head>
▼ <body class="wide comments example" data-new-gr-c-s-check-loaded="14.983.0">
    <a name="top" id="top"></a>
  ▶ <div class="fw-background">…</div>
  ▼ <div class="fw-container">
    ▶ <div class="fw-header">…</div>
    ▶ <div class="fw-nav">…</div>
    ▼ <div class="fw-body">
      ▼ <div class="content">
          <h1 class="page_title">Zero configuration</h1>
        ▶ <div class="info" style="min-height: 8em;">…</div>
        ▼ <div class="demo-html">
          ▼ <div id="example_wrapper" class="dataTables_wrapper">
            ▶ <div class="dataTables_length" id="example_length">…</div>
            ▶ <div id="example_filter" class="dataTables_filter">…</div>
            ▼ <table id="example" class="display dataTable" style="width: 100%;" role="grid" aria-describedby="example_info">
              ▶ <thead>…</thead>
              ▼ <tbody>
                ▼ <tr role="row" class="odd">
                    <td class="sorting_1">Airi Satou</td>
                    <td>Accountant</td>
                    <td>Tokyo</td>
                    <td>33</td>
                    <td>2008/11/28</td>
                    <td>$162,700</td>
                  </tr>
                ▼ <tr role="row" class="even">
                    <td class="sorting_1">Angelica Ramos</td>
                    <td>Chief Executive Officer (CEO)</td>
                    <td>London</td>
                    <td>47</td>
                    <td>2009/10/09</td>
                    <td>$1,200,000</td>
                  </tr>
                ▼ <tr role="row" class="odd">
                    <td class="sorting_1">Ashton Cox</td>
                    <td>Junior Technical Author</td>
                    <td>San Francisco</td>
                    <td>66</td>
                    <td>2009/01/12</td>
                    <td>$86,000</td>
                  </tr>
```

Web scraping

```
>>> import requests
>>> from bs4 import BeautifulSoup
>>> import pandas as pd
```

```
>>> url = 'https://datatables.net/examples/basic_init/
    zero_configuration.html'
```

```
>>> response = requests.get(url)
>>> soup = BeautifulSoup(response.text, 'html.parser')
>>> info = soup.find_all('tr')
```

Web scraping

```
>>> print(info)
[<tr>
<th>Name</th>
<th>Position</th>
<th>Office</th>
<th>Age</th>
<th>Start date</th>
<th>Salary</th>
</tr>, <tr>
<td>Tiger Nixon</td>
<td>System Architect</td>
<td>Edinburgh</td>
<td>61</td>
<td>2011/04/25</td>
<td>$320,800</td>
</tr>, <tr>
<td>Cedric Kelly</td>
```

Web scraping

```
>>> df = pd.DataFrame(columns=['name', 'position', 'office',
                               'age', 'start', 'salary'])
```

```
>>> for i, item in enumerate(info):
...     if i != 0 and i != len(info)-1:
...         data_point = item.find_all('td')
...         row = []
...         for value in data_point:
...             row.append(value.text)
...         df.loc[i-1] = row
```

Table of Contents

1. Objetivos de esta clase
2. Descargar con un click
3. Web scraping
4. PDF scraping
5. Base de datos relacionales (tipo SQL)
6. Conexión a una API

PDF Scraping

Utilizaremso el siguiente PDF: **Boletín IFE - ANSES**
Utilizamos el paquete PyDF2

```
>>> import PyPDF2
```

Utilizamos el metodo 'PdfFileReader' y vamos imprimir el contenido leido en cada pagina con un loop y los metodos 'getPage' y 'extractText'

```
>>> pdf_file = open('boletin_ife.pdf', 'rb')
>>> pdfReader = PyPDF2.PdfFileReader(pdf_file)
>>> paginas = pdfReader.numPages
>>> for i in range(paginas):
...     print("Contenido de la pagina {}:".format(i))
...     pagina = pdfReader.getPage(i)
...     print(pagina.extractText())
```

PDF Scraping

Para extraer las tablas del PDF vamos a usar el paquete tabula

```
>>> import tabula
>>> import pandas as pd
```

Utilizamos el metodo 'read_pdf', le vamos a solicitar la tabla de la pagina 7 y que la devuelva como dataframe

```
>>> lst_of_df = tabula.read_pdf('boletin_ife.pdf',
    output_format='dataframe', pages=7)
```

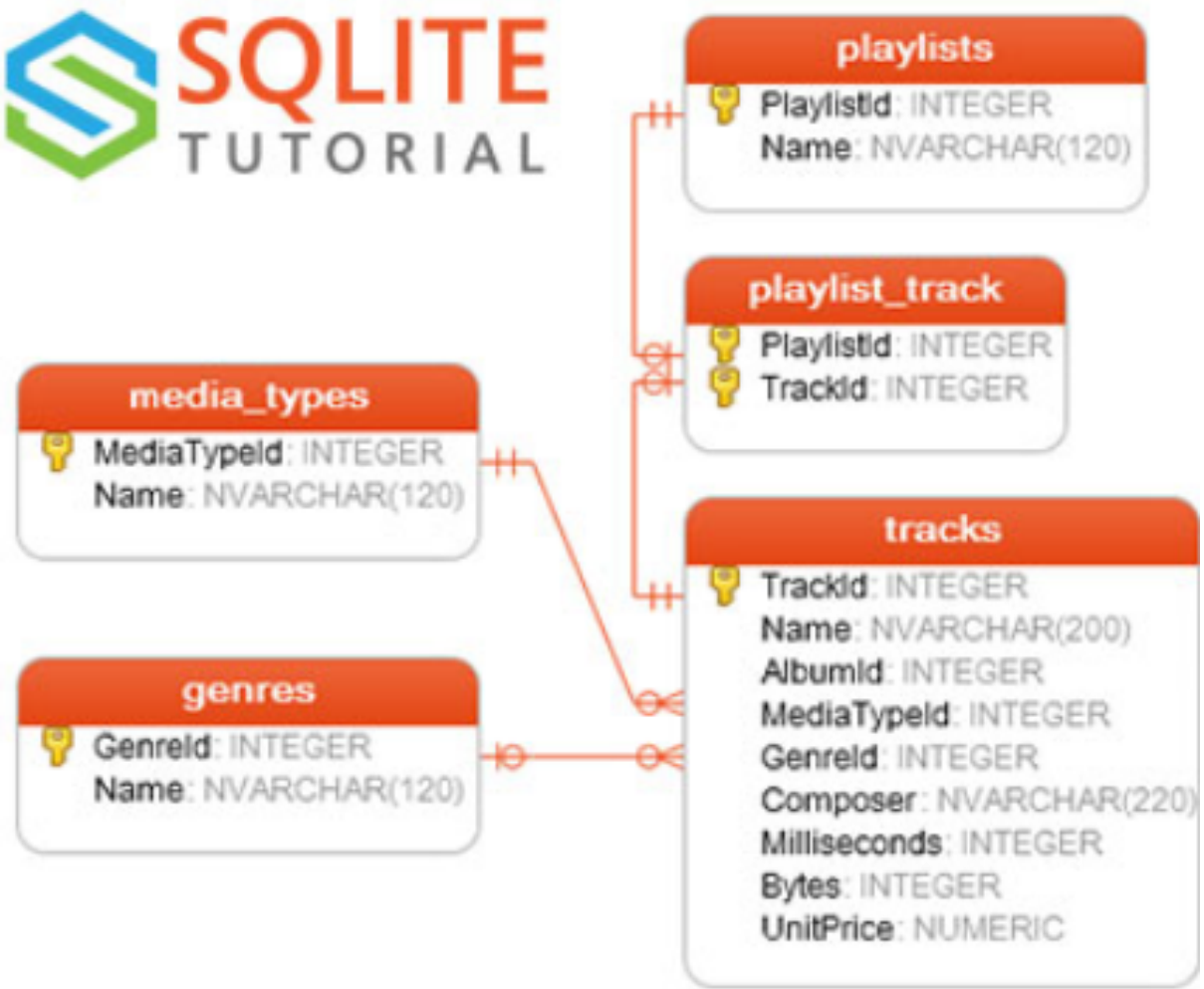
```
>>> tabla_p7 = lst_of_df[0]
>>> tabla_p7.columns = ['tipo_de_hogar','pobreza', '
    indigencia']
```

Table of Contents

1. Objetivos de esta clase
2. Descargar con un click
3. Web scraping
4. PDF scraping
5. Base de datos relacionales (tipo SQL)
6. Conexión a una API

Base de Datos relacionales

Figure: Base de datos relacional



fuente: https://www.sqlitetutorial.net/

Base de Datos relacionales

Figure: SQLite: No requiere servidor

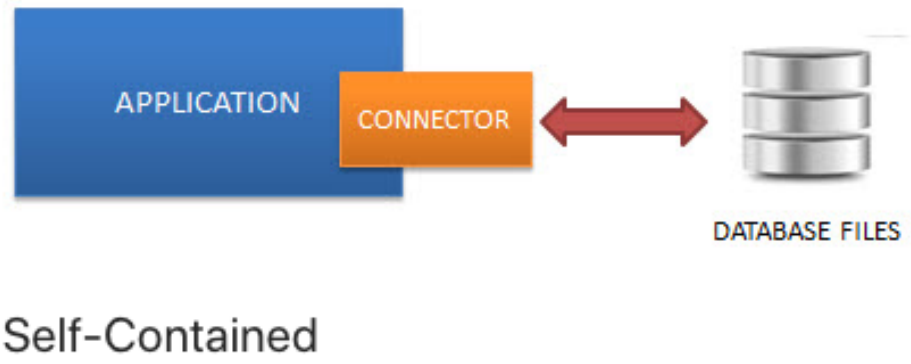
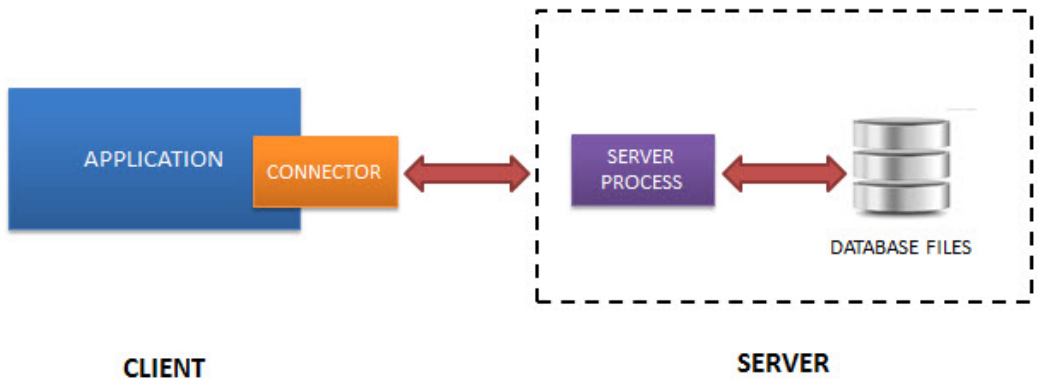


Figure: PostgreSQL: Requiere un servidor de BD



fuente: sqlite vs postgresql

Base de Datos relacionales

Figure: SQLite: Comandos de selección de datos

```
SELECT *                    #Que se quiere seleccionar
FROM hogares_uni            #De que tabla
WHERE total > 10000         #Condición
ORDER BY provincia;         #Variable que ordena
```

Figure: SQLite: Comandos de edición de tablas

```
CREATE                      #Para crear una tabla
DROP                        #Para eliminar una tabla
INSERT                      #Para agregar obs. a la tabla
```

Base de Datos relacionales

Utilizamos el paquete sqlite3

```
>>> import sqlite3
```

Iniciamos la conexión a la base de datos con el metodo 'connect' y creamos un objeto cursor para poder comenzar a ejecuter los comandos de SQL:

```
>>> conn = sqlite3.connect('IFE.db')
>>> c = conn.cursor()
```

Creo la tabla, inserto una observacion a la tabla

```
>>> c.execute('''CREATE TABLE provincias (provincia text,
                         prov_cod INTEGER PRIMARY KEY)''')
>>> c.execute("INSERT INTO provincias VALUES ('Buenos Aires
    ', 1)")
```

Base de Datos relacionales

Insertamos observaciones en bloque a la tabla

```
>>> provincias = [('CABA', 2),
                  ('Catamarca', 3),
                  ('Chaco', 4)]
>>> c.executemany('INSERT INTO provincias VALUES (?,?)',
    provincias)
```

Guardamos (commit) los cambios y cerramos la conexión

```
>>> conn.commit()
>>> conn.close()
```

Table of Contents

1. Objetivos de esta clase
2. Descargar con un click
3. Web scraping
4. PDF scraping
5. Base de datos relacionales (tipo SQL)
6. Conexión a una API

Conexión a una API

- Las API (application programming interface) son una interfaz informática que define las interacciones entre múltiples intermediarios de software.
- Define los tipos de llamadas o solicitudes que se pueden realizar, cómo hacerlas, los formatos de datos que deben usarse, las convenciones a seguir, etc.
- También puede proporcionar mecanismos de extensión para que los usuarios puedan extender la funcionalidad existente de varias formas y en diferentes grados.
- Las API muchas veces requieren autenticación para garantizar la seguridad de los datos.

Cierre

Gracias

Esperamos sus dudas en Slack